

Translating Ancient Akkadian: A Comparative Study of ByT5 Architectures

CTI / Semester Project

Teodor Mitrica
Faculty of Mathematics and Computer Science
`teodor.mitrica@s.unibuc.ro`

June 24, 2026

Abstract

This project investigates the application of ByT5 models for translating ancient Akkadian into English, as part of the Deep Past Challenge competition on Kaggle. This report details the data preprocessing, training strategies on Kaggle, and a comparative study of three model capacities: ByT5-Small, Base, and Large. I demonstrate that while smaller models excel at administrative formulas, and the Base model offers the best generalization, massive models like ByT5-Large are highly susceptible to overfitting on small datasets, degrading into lookup dictionaries.

1 Introduction

The problem I am trying to solve is the automatic translation of Akkadian transliterated texts into modern English.

For this project, I handled the entire machine learning pipeline: data cleaning, tokenization strategy, model training, hyperparameter tuning, hardware optimization, and final performance analysis.

A summary of the experiments tried includes:

- Fine-tuning **ByT5-Small** (~300M parameters) to establish a baseline.
- Fine-tuning **ByT5-Base** (~580M parameters) to improve contextual translation.
- Training **ByT5-Large** (~1.2B parameters) using BLEU metric validation, which timed out due to hardware constraints.
- A high-speed complete training of **ByT5-Large** using only validation loss to study capacity scaling.

My approach relies on the ByT5 architecture. I chose this specific architecture because Akkadian transliteration relies heavily on special diacritics, subscripts, and fragmented words (due to broken tablets), making traditional subword tokenizers inefficient. By operating at the byte level, ByT5 eliminates out-of-vocabulary issues.

2 Approach

2.1 Data Processing and Environment

The project code and resources are maintained in a dedicated repository [Insert GitHub/Drive Link Here]. Development was conducted entirely on Kaggle Notebooks using PyTorch and HuggingFace Transformers.

I implemented a custom cleaning function to normalize Akkadian transliteration. This included standardizing accented characters, translating Unicode subscripts to regular digits, mapping decimal representations to fraction characters, and unifying destroyed tablet sections into a standard ‘<gap>’ token. A post-processing pipeline was also used to fix punctuation spacing and capitalization in the generated English text.

2.2 Training Strategies

Training large language models on Kaggle presented severe hardware limitations (16GB VRAM and a strict 12-hour session limit).

To train the **Large** model, I had to employ several optimization tricks:

1. **FP16 Precision:** Crucial to fit the model weights into memory and speed up matrix multiplications.
2. **Gradient Accumulation and Checkpointing:** Allowed simulating larger batch sizes without triggering CUDA Out-Of-Memory errors.
3. **Adafactor Optimizer:** Used instead of AdamW to significantly reduce memory footprint.
4. **Infrastructure Bypass:** Since the 12-hour limit killed the Large model’s auto-sync, I zipped the checkpoint from the temporary ‘/kaggle/working’ directory and uploaded it as a static Kaggle Dataset.

2.3 Evaluation and Comparative Analysis

The models were evaluated using the competition score on the associated test set. The results are summarized in Table 1.

Model	Parameters	BLEU Score
ByT5-Small	~300M	23.94
ByT5-Base	~580M	29.71
ByT5-Large (Partial)	~1.2B	26.78
ByT5-Large (Fully)	~1.2B	20.57

Table 1: Competition Test Set Results

A sentence-level BLEU analysis revealed distinct behaviors based on model capacity:

- **The Small Model:** Achieved baseline performance (23.94 BLEU) by successfully capturing repetitive administrative formulas and legal receipts (e.g., *"Seal of X, to Y"*). However, due to its constrained parameter size, it suffered from “Early Stopping by Attention Failure” on long, complex letters, generating only the formulaic greeting and entirely dropping the subsequent narrative body.
- **The Base Model:** Achieved the highest score (29.71 BLEU) by successfully balancing grammatical abstraction with context memory. It represented the optimal capacity for this corpus size.

- **The Large Model - Partially Trained:** Halted prematurely at epoch 5 due to the 12-hour hardware constraint, this variant achieved a score of 26.78 BLEU but exhibited a unique “Short-Circuiting” phenomenon on long texts. Optimized via generative BLEU validation, the model learned a protective strategy: instead of risking syntactic collapse on unseen narrative chunks, it extracted only 3-4 high-confidence entities and immediately emitted the End-of-Sequence token.
- **The Large Model - Fully Trained:** Deprived of generative regularization and optimized purely on Validation Loss for 10 epochs, the model degraded into lookup dictionary. On unseen validation data, it failed due to two distinct failure modes: “Rigid Truncation”, where generation stopped abruptly the moment the test context diverged from memorized training keys, and “Template Splicing”, where encountering a familiar opening forced the decoder to copy clauses memorized from the training set.

3 Conclusions

This project demonstrated that model capacity alone does not guarantee better translation quality, especially when training data is limited. Among the evaluated architectures, ByT5-Base achieved the best balance between memorization and generalization, obtaining the highest competition score.

Beyond the competition results, this project provided valuable practical experience with the ByT5 architecture, byte-level text processing, and the challenges of fine-tuning large language models under hardware constraints.

What could have been done differently?

If I were to start over, I would implement data augmentation techniques to artificially increase the effective size of the training corpus and reduce overfitting in larger models. Furthermore, I would incorporate the additional Akkadian texts provided by the competition organizers that were not included in the structured training set. Converting and integrating these resources into the training pipeline would increase the number of available samples and potentially improve the model’s ability to generalize to unseen texts.

Note: Top 3 solutions for the competition

1. <https://kaggle.com/competitions/deep-past-initiative-machine-translation/writeups/dpc-1st-data-quality-dictates-everything>
2. <https://kaggle.com/competitions/deep-past-initiative-machine-translation/writeups/2nd-place-data-centric-akkadian-nmt>
3. <https://www.kaggle.com/competitions/deep-past-initiative-machine-translation/writeups/3rd-synthetic-data-to-teach-oa-fundamentals>